

S01 - Text to Audio

POST <https://api.groq.com/openai/v1/audio/speech> Send

Docs Params Authorization Headers 12 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐ Schema Beautify

```
1 {
2   "model": "canopylabs/orpheus-v1-english",
3   "input": "Here are the rules for the two-player Deal game. First, both players start the match with
      a maximum of one hundred and sixty points. Second, the game consists of exactly two deals.
      Please note that the drop option is not available in Deal games. Third, at the end of the
      second deal, the player with the highest points will be declared the winner. Finally, if there
      is a tie at the end of the second deal, an extra tie-breaker deal will be played to decide the
      ultimate winner",
4   "voice": "diana",
5   "response_format": "wav",
6   "speed": 1.5
7 }
```

Body Cookies 1 Headers 21 Test Results 200 OK • 4.78 s • 1.33 MB • Save Response ...

Hex Preview Visualize

0:00 / 0:29

Body Request:

```
{
  "model": "canopylabs/orpheus-v1-english",
  "input": "Here are the rules for the two-player Deal game.
First, both players start the match with a maximum of one
hundred and sixty points. Second, the game consists of exactly
two deals. Please note that the drop option is not available in
```

Deal games. Third, at the end of the second deal, the player with the highest points will be declared the winner. Finally, if there is a tie at the end of the second deal, an extra tie-breaker deal will be played to decide the ultimate winner",

```
"voice": "diana",  
"response_format": "wav",  
"speed" : 1.5  
}
```

S02- Multilingual

GenAI-June-HomeAssignment > S02- MultiLingual

SaveShare

POSThttps://api.groq.com/openai/v1/chat/completionsSend

DocsParamsAuthorizationHeaders 10BodyScriptsSettingsCookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

SchemaBeautify

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     { "role": "system", "content": "You are a professional translator. Translate to Hindi. Preserve
5       tone, idioms, and meaning. Return only the translation." },
6     { "role": "user", "content": "Here are the rules for the two-player Deal game. First, both players
7       start the match with a maximum of one hundred and sixty points. Second, the game consists of
8       exactly two deals. Please note that the drop option is not available in Deal games. Third, at
9       the end of the second deal, the player with the highest points will be declared the winner.
10      Finally, if there is a tie at the end of the second deal, an extra tie-breaker deal will be
11      played to decide the ultimate winner" }
12   ],
13   "temperature": 0.3,
14   "max_completion_tokens": 2000
15 }
```

BodyCookies 1Headers 20Test Results200 OK • 964 ms • 1.47 KBSave Response

JSONPreviewVisualize

```
1 {
2   "id": "chatcmpl-eb10736d-1b8a-4b91-bfb9-1229acc47145",
3   "object": "chat.completion",
4   "created": 1781799616,
5   "model": "llama-3.3-70b-versatile",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "दो खिलाड़ियों के डील गेम के नियम इस प्रकार हैं। पहले, दोनों खिलाड़ी मैच में
12        शुरू होते हैं और प्रत्येक डील में अधिकतम 160 प्वाइंट्स का उपयोग कर सकते हैं। दूसरे, गेम दो डील से
13        शुरू होता है। प्रत्येक डील के बाद, प्वाइंट्स काउंट किया जाता है। तीसरे, दो डील के बाद,
14        जिस खिलाड़ी के पास अधिकतम प्वाइंट्स हैं, वह जीतता है। यदि दोनों खिलाड़ी बराबर
15        प्वाइंट्स के साथ हैं, तो एक अतिरिक्त डील खेली जाती है।"
16      }
17     ]
18   }
19 }
```

Request:

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    { "role": "system", "content": "You are a professional translator. Translate to Hindi. Preserve tone, idioms, and meaning. Return only the translation." },
    { "role": "user", "content": "Here are the rules for the two-player Deal game. First, both players start the match with a maximum of one"
  ]
}
```

```
hundred and sixty points. Second, the game consists of exactly two
deals. Please note that the drop option is not available in Deal
games. Third, at the end of the second deal, the player with the
highest points will be declared the winner. Finally, if there is a
tie at the end of the second deal, an extra tie-breaker deal will be
played to decide the ultimate winner"}
],
"temperature": 0.3,
"max_completion_tokens": 2000
}
```

Response:

```
{
  "id":
"chatcmpl-eb10736d-1b8a-4b91-bfb9-1229acc47145",
  "object": "chat.completion",
  "created": 1781799616,
  "model": "llama-3.3-70b-versatile",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "दो खिलाड़ियों के डील गेम के नियम इस
प्रकार हैं। पहले, दोनों खिलाड़ी मैच में एक सौ साठ बिंदुओं की
अधिकतम सीमा के साथ शुरू करते हैं। दूसरा, गेम ठीक दो डील से
बना है। कृपया ध्यान दें कि डील गेम्स में ड्रॉप विकल्प उपलब्ध
नहीं है। तीसरा, दूसरी डील के अंत में, सबसे अधिक बिंदुओं वाले
```

खिलाड़ी को विजेता घोषित किया जाएगा। अंत में, यदि दूसरी डील के अंत में टाई होती है, तो अंतिम विजेता का फैसला करने के लिए एक अतिरिक्त टाई-ब्रेकर डील खेला जाएगा"

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.05024435,
  "prompt_tokens": 161,
  "prompt_time": 0.007955618,
  "completion_tokens": 222,
  "completion_time": 0.646486762,
  "total_tokens": 383,
  "total_time": 0.65444238
},
"usage_breakdown": null,
"system_fingerprint": "fp_dae98b5ecb",
"x_groq": {
  "id": "req_01kvdrgzcxeeaya12jsdggqqdce",
  "seed": 1550855202
},
"service_tier": "on_demand"
}
```

S03 - GenerateAPITestCasefromSwaggerCode

Request:

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I - INSTRUCTION\nGenerate comprehensive API test cases designed specifically to be automated using Rest Assured.\n\nMANDATORY RULES:\n\nDo NOT generate Java or Rest Assured code. Generate the test scenarios and test cases only.\n\nInclude positive, negative, and edge-case scenarios for each endpoint.\n\nDetail the specific HTTP methods, endpoints, and required headers (e.g., Content-Type, Authorization).\n\nSpecify the exact JSON payloads required for the requests (create mock data relevant to a subscription service).\n\nClearly define the Expected Results, including the exact HTTP Status Codes and specific JSON response fields to assert (using JsonPath logic).\n\nC - CONTEXT\nThese test cases will be used as the blueprint by QA Automation Engineers to build a robust Rest Assured framework. The test cases must clearly map out exactly what needs to be verified in the API response so the engineer knows exactly what assertions to write in their code.\n\nP - PERSONA\nYou are a Senior API Test Analyst known for writing meticulous, edge-case-inclusive, and automation-ready API test documentation.\n\nO - OUTPUT\nOutput the test cases in a clean, structured format (like a Markdown table or clearly separated blocks).\n\nFor each test case, include:\n\nTest Case ID: (e.g., TC_001)\n\nScenario Description: - HTTP Method: - Endpoint/Path: - Headers / Auth: - Request Payload: (Include sample JSON)\n\nExpected Status Code: - Response Assertions: (What exact fields or data types to verify in the response body)\n\nT - TARGET\nAPI:\n\nAPI Name: Subscriptions API\n\nBase URI:
```

```
https://gateway-qa-8.jungleerummyqa.com\nResource to Test: Subscriptions\nAvailable  
Endpoints:\n\nCreate Subscription: POST /subs-api/v1/subscriptions\n\nGet  
Subscription: GET /subs-api/v1/subscriptions/{subscriptionId}\n\nUpdate Subscription:  
PUT (or PATCH) /subs-api/v1/subscriptions/{subscriptionId}\n\nDelete Subscription:  
DELETE /subs-api/v1/subscriptions/{subscriptionId}"  
    }  
  ]  
}
```

Response

```
{  
  "id":  
    "chatcmpl-b4e0c759-9199-4658-b63d-6a6ccd  
    7051ad",  
  "object": "chat.completion",  
  "created": 1781802188,  
  "model": "openai/gpt-oss-120b",  
  "choices": [  
    {  
      "index": 0,  
      "message": {  
        "role": "assistant",  
        "content": "#  Subscriptions API  
– Test-Case Blueprint \n\n**Base URI**:  
`https://gateway-qa-8.jungleerummyqa.com`
```

\n**Common Headers** \n\n| Header | Value
(when required)

\n|-----|-----|\n| `Content-Type`
| `application/json` |\n| `Accept` |
`application/json` |\n| `Authorization` | `Bearer
<valid-jwt-token>` (replace with a token that
has the **subscriptions** scope) \n\n>

Note – For every negative-auth scenario
use an **invalid**, **expired**, or **missing**
token. \n\n--- \n\n### 1 Create Subscription –

`POST /subs-api/v1/subscriptions`\n\n| Test
Case ID | Scenario | HTTP Method | Endpoint
| Headers | Request Payload | Expected
Status Code | Response Assertions

\n|-----|-----|-----|-----|-----
---|-----|-----|-----
-----|\n| **SUB_CR_001** | **Happy path –
create a new active subscription** | **POST** |
`/subs-api/v1/subscriptions` | **Content-Type,**
Accept, Authorization (valid) | ``json {
`"userId": "U12345", "planId":


```
\\"PLAN_BASIC\\", \\"startDate\\":
\\"2024-07-01\\", \\"endDate\\": \\"2025-07-01\\",
\\"paymentMethod\\": { \\"type\\": \\"credit_card\\",
\\"cardNumber\\": \\"4111111111111111\\",
\\"expiry\\": \\"12/30\\", \\"cvv\\": \\"123\\" },
\\"metadata\\": { \\"referralCode\\":
\\"FRIEND2024\\" } }`` | **201 Created** |
*Body is JSON* <br>• `$.subscriptionId` –
**exists**, non-empty string (UUID format)
<br>• `$.status` = `\\"ACTIVE\\"` <br>•
`$.planId` = `\\"PLAN_BASIC\\"` <br>•
`$.userId` = `\\"U12345\\"` <br>• `$.startDate` =
`\\"2024-07-01\\"` <br>• `$.endDate` =
`\\"2025-07-01\\"` <br>• `$.createdAt` –
present, ISO-8601 timestamp <br>•
`$.paymentMethod.type` = `\\"credit_card\\"`
|\\n| **SUB_CR_002** | Missing mandatory
field **planId** | POST |
`/subs-api/v1/subscriptions` | Same as above
| ``json { \\"userId\\": \\"U12345\\", \\"startDate\\":
\\"2024-07-01\\", \\"endDate\\": \\"2025-07-01\\",
```

```
"paymentMethod": { "type": "credit_card",
"cardNumber": "4111111111111111",
"expiry": "12/30", "cvv": "123" } }` |
**400 Bad Request** | `$.error.code` =
`"VALIDATION_ERROR"` <br>•
`$.error.message` contains `"planId is
required"` |`n| **SUB_CR_003** | Invalid date
format (ISO-8601 required) | POST |
`/subs-api/v1/subscriptions` | Same as above
| ``json { "userId": "U12345", "planId":
"PLAN_BASIC", "startDate":
"07/01/2024", "endDate": "07/01/2025",
"paymentMethod": { "type": "credit_card",
"cardNumber": "4111111111111111",
"expiry": "12/30", "cvv": "123" } }` |
**400 Bad Request** | `$.error.code` =
`"VALIDATION_ERROR"` <br>•
`$.error.details[?(@.field=='startDate')].message` contains `"must be in ISO-8601 format"`
|`n| **SUB_CR_004** | Duplicate subscription
(same userId + planId + startDate) | POST |
```

`/subs-api/v1/subscriptions` | Same as above
| *Same payload as SUB_CR_001* (run after
a successful creation) | **409 Conflict** |

`\$.error.code` =

`"DUPLICATE\_SUBSCRIPTION"`
•

`\$.error.message` = `"A subscription with the
same user, plan and start date already
exists"` |
`SUB\_CR\_005` | Unauthorized
– missing JWT | POST |

`/subs-api/v1/subscriptions` | Content-Type,
Accept (no Authorization) | Valid payload
(same as SUB_CR_001) | **401

Unauthorized** | `\$.error.code` =

`"UNAUTHORIZED"`
•

`\$.error.message` = `"Authentication token
required"` |
`SUB\_CR\_006` | Forbidden
– token lacks required scope | POST |

`/subs-api/v1/subscriptions` | Authorization
with a token that **does not** contain

`subscriptions:create` scope | Valid payload |

403 Forbidden | `\$.error.code` =

`"FORBIDDEN"`
• `\$.error.message`
contains `"insufficient permissions"` |
`\*\*SUB\_CR\_007\*\*` | Payload exceeds size
limit (large metadata) | POST |
`/subs-api/v1/subscriptions` | Same as
happy-path | ``json { "userId": "U12345",
"planId": "PLAN_BASIC", "startDate":
"2024-07-01", "endDate": "2025-07-01",
"paymentMethod": { "type": "credit_card",
"cardNumber": "4111111111111111",
"expiry": "12/30", "cvv": "123" },
"metadata": { "notes": "<string of 10 KB>"
} } `` | ****413 Payload Too Large**** (if server
enforces) | `\$.error.code` =
`"PAYLOAD\_TOO\_LARGE"` |
****SUB_CR_008**** | Unsupported Media Type
– send XML instead of JSON | POST |
`/subs-api/v1/subscriptions` | `Content-Type:
application/xml`, Authorization valid |
`<subscription><userId>U12345</userId>...</
subscription>` | ****415 Unsupported Media**

Type** | `\$.error.code` =
`"UNSUPPORTED\_MEDIA\_TYPE"` |
SUB_CR_009 | Edge – create
subscription with **future** startDate far in the
future (e.g., 10 years) | POST |
`/subs-api/v1/subscriptions` | Same as
happy-path | ``json { "userId": "U12345",
"planId": "PLAN_PREMIUM", "startDate":
"2034-01-01", "endDate": "2035-01-01",
"paymentMethod": { "type":
"bank_transfer", "accountNumber":
"12345678", "routingNumber":
"87654321" } } `` | **201 Created** (if
allowed) | Same assertions as happy-path,
plus verify `\$.status` = `"PENDING"` if
business rule sets future start as pending |
SUB_CR_010 | Null values for optional
fields (metadata omitted) | POST |
`/subs-api/v1/subscriptions` | Same as
happy-path | ``json { "userId": "U12345",
"planId": "PLAN_BASIC", "startDate":

```
\ "2024-07-01\ ", \ "endDate\ ": \ "2025-07-01\ ",  
\ "paymentMethod\ ": { \ "type\ ": \ "credit_card\ ",  
\ "cardNumber\ ": \ "4111111111111111\ ",  
\ "expiry\ ": \ "12/30\ ", \ "cvv\ ": \ "123\ " } } `` |
```

****201 Created**** | Verify `$.metadata` is either
****null**** or ****absent****; other fields as usual

| \n\n--- \n\n### 2 Get Subscription – `GET
/subs-api/v1/subscriptions/{subscriptionId}`

\n\n Test Case ID	Scenario	HTTP Method	Endpoint	Headers	Path Variable	Expected Status Code	Response Assertions
--------------------	----------	-------------	----------	---------	---------------	----------------------	---------------------

\n -----	-----	-----	-----	-----	-----	-----	-----
---	-----	-----	-----	-----	-----	-----	-----

----| \n| ****SUB_GT_001**** | Happy path –
retrieve existing subscription | GET |

`/subs-api/v1/subscriptions/{subscriptionId}` |
Accept, Authorization (valid) | ``<id returned
from SUB_CR_001>`` | ****200 OK**** |

`\$.subscriptionId` = ``<same id>``
•

`\$.status` present (e.g., ``"ACTIVE"`)
•

`\$.planId`, `\$.userId`, `\$.startDate`,

`\$.endDate` all non-null
• `\$.createdAt`
ISO-8601 | \n| **SUB_GT_002** | Subscription
not found | GET |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Same as above | `non-existent-uuid-0000` |
404 Not Found | `\$.error.code` =
`"SUBSCRIPTION\_NOT\_FOUND"`
•
`\$.error.message` contains the supplied id | \n|
SUB_GT_003 | Invalid UUID format in
path | GET |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Same as above | `12345-invalid` | **400 Bad
Request** | `\$.error.code` =
`"INVALID\_PATH\_PARAMETER"`
•
`\$.error.message` contains `"subscriptionId
must be a valid UUID"` | \n| **SUB_GT_004**
| Unauthorized – missing token | GET |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Accept only (no Authorization) | Valid id |
401 Unauthorized | `\$.error.code` =
`"UNAUTHORIZED"` | \n| **SUB_GT_005** |

Forbidden – token without read scope | GET |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Authorization (token lacks
`subscriptions:read`) | Valid id | **403
Forbidden** | `\$.error.code` =
`"FORBIDDEN"` |
`\*\*SUB\_GT\_006\*\*` |
Conditional GET – use `If-None-Match` with
correct ETag | GET |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Accept, Authorization, `If-None-Match`:
`"<etag-from-previous-GET>"` | Valid id |
304 Not Modified | No body (or empty);
verify header `ETag` unchanged |
`\*\*SUB\_GT\_007\*\*` | Conditional GET – stale
ETag (resource changed) | GET |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Accept, Authorization, `If-None-Match`:
`"stale-etag"` | Valid id | **200 OK** |
Response body as normal; new `ETag`
header differs from request value |
`\*\*SUB\_GT\_008\*\*` | Edge – retrieve

subscription with ****large metadata**** (10 KB) |
GET |

`/subs-api/v1/subscriptions/{subscriptionId}` |
Same as happy-path | Id of a subscription
created with large metadata (SUB_CR_007) |
****200 OK**** | Verify `\$.metadata.notes` length
≈ 10 KB; other fields unchanged |
|

****SUB_GT_009**** | Accept header set to an
unsupported media type | GET |

`/subs-api/v1/subscriptions/{subscriptionId}` |
`Accept: application/xml`, Authorization valid |
Valid id | ****406 Not Acceptable**** |

`\$.error.code` = `"NOT\_ACCEPTABLE"` |
|

****SUB_GT_010**** | Request with additional
unknown query parameter (should be
ignored) | GET |

`/subs-api/v1/subscriptions/{subscriptionId}?d
ebug=true` | Same as happy-path | Valid id |
****200 OK**** | Same assertions as happy-path;
ensure extra param does not break response
|
|

--- \n\n## 3 Update Subscription – `PUT

```

/subs-api/v1/subscriptions/{subscriptionId}`
\n*(If the API supports PATCH, add
analogous cases for PATCH – they are
omitted for brevity.)* \n\n| Test Case ID |
Scenario | HTTP Method | Endpoint | Headers
| Path Variable | Request Payload | Expected
Status Code | Response Assertions
|\n|-----|-----|-----|-----|-----
---|-----|-----|-----|-----|
-----|\n| **SUB_UP_001** | Happy
path – change plan & endDate | PUT |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Content-Type, Accept, Authorization (valid) |
Id from **SUB_CR_001** | ``json { "planId":
"PLAN_PREMIUM", "endDate":
"2025-12-31" }`` | **200 OK** |
`$.subscriptionId` unchanged <br>• `$.planId`
= "PLAN_PREMIUM" <br>• `$.endDate` =
"2025-12-31" <br>• `$.status` stays
"ACTIVE" <br>• `$.updatedAt` present,
newer than `createdAt` |\n| **SUB_UP_002**

```

| Attempt to modify ****immutable**** field
`userId` | PUT |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Same as above | Same id | ``json { \"userId\":
\"U99999\", \"planId\": \"PLAN_BASIC\" }`` |
****400 Bad Request**** | `\$.error.code` =
`\"IMMUTABLE\_FIELD\"`
•
`\$.error.message` contains `\"userId cannot
be changed\"` |\\n| ****SUB_UP_003**** | Full
payload missing required `planId` (PUT
expects whole resource) | PUT |
`/subs-api/v1/subscriptions/{subscriptionId}` |
Same as above | Same id |,

\"reasoning\": \"We need to produce
test cases, no code. Provide comprehensive
positive, negative, edge. Include headers,
payload, expected status, assertions with
JsonPath. Provide table or blocks.\\n\\nWe'll
list many test cases for each endpoint.\\n\\nWill
include authentication header maybe Bearer
token.\\n\\nWill include Content-Type:

application/json.\n\nWill include scenarios like valid create, missing fields, invalid data types, duplicate subscription, unauthorized, etc.\n\nFor Get: valid ID, non-existing ID, invalid format ID, unauthorized.\n\nUpdate: full update with valid data, partial update, attempt to change immutable field, invalid payload, concurrency (ETag) maybe.\n\nDelete: valid delete, delete non-existing, unauthorized, trying to delete already cancelled subscription.\n\nEdge cases: large payload, special characters in fields, maximum length, zero-length arrays.\n\nWill define JSON payload sample: subscription with fields like subscriptionId (maybe generated), userId, planId, startDate, endDate, status, paymentMethod, metadata.\n\nDefine assertions: status code, response JSON fields like \"subscriptionId\", \"status\", \"createdAt\", \"planId\", etc. Use JsonPath like \$.subscriptionId, \$.status,

\$.error.message.\n\nWill produce test case IDs like SUB_001 etc.\n\nWill output in markdown tables per endpoint.\n\nLet's craft."

```
    },
    "logprobs": null,
    "finish_reason": "length"
  }
],
"usage": {
  "queue_time": 0.282034581,
  "prompt_tokens": 503,
  "prompt_time": 0.022572478,
  "completion_tokens": 3072,
  "completion_time": 6.443996242,
  "total_tokens": 3575,
  "total_time": 6.46656872,
  "completion_tokens_details": {
    "reasoning_tokens": 262
  }
},
"usage_breakdown": null,
```

```
"system_fingerprint": "fp_90620edd96",  
"x_groq": {  
  "id": "req_01kvdzt8tgfs8tc6j9eyff0tve",  
  "seed": 1659782804  
},  
"service_tier": "on_demand"  
}
```